

سوال:

بخش اول: طراحی کلاس و وراثت (Inheritance)

فرض کنید که شما یک سیستم مدیریت کتابخانه را توسعه می‌دهید. در این سیستم، کتاب‌ها دارای ویژگی‌هایی مانند عنوان، نویسنده، سال انتشار و نوع کتاب (مثلاً علمی، داستانی، تاریخی و غیره) هستند. این سیستم باید به گونه‌ای باشد که امکان اضافه کردن انواع مختلف کتاب‌ها را داشته باشد.

1. **کلاس پایه کتاب (Book)** را طراحی کنید. این کلاس باید شامل ویژگی‌های عنوان، نویسنده، سال انتشار و نوع کتاب باشد. علاوه بر این، متدی به نام `get_details` ایجاد کنید که جزئیات کتاب را به صورت رشته‌ای برگرداند.
2. یک کلاس **کتاب علمی (ScientificBook)** ایجاد کنید که از کلاس `Book` ارث‌بری کند. این کلاس باید علاوه بر ویژگی‌های کلاس پایه، یک ویژگی اضافی به نام "موضوع" داشته باشد و همچنین متدی به نام `get_details` را بازنویسی (`override`) کند تا موضوع را نیز در جزئیات کتاب نمایش دهد.
3. یک کلاس **کتاب داستانی (FictionBook)** ایجاد کنید که از کلاس `Book` ارث‌بری کند. این کلاس باید یک ویژگی اضافی به نام "رده سنی" داشته باشد و متدی به نام `get_details` را بازنویسی کند تا رده سنی را نیز به جزئیات کتاب اضافه کند.

بخش دوم: Extension Method

- اکنون که کلاس‌های مربوط به کتاب‌ها را طراحی کردید، به شما گفته شده است که به قابلیت‌های اضافی برای بررسی موجود بودن کتاب در کتابخانه و همچنین مرتب‌سازی لیست کتاب‌ها بر اساس سال انتشار نیاز داریم. برای این کار از **Extension Method** استفاده کنید.
1. یک **متد Extension** برای کلاس‌های مختلف کتاب ایجاد کنید که بررسی کند آیا کتاب در مجموعه کتابخانه موجود است یا خیر. به این صورت که یک لیست از کتاب‌ها به عنوان

ورودی می‌گیرد و اگر کتاب موجود باشد، مقدار **True** و در غیر این صورت **False** را برمی‌گرداند.

2. **یک متد Extension** دیگر بنویسید که لیست کتاب‌ها را بر اساس سال انتشار مرتب کند. این متد باید قابلیت مرتب‌سازی صعودی و نزولی را داشته باشد و بسته به پارامتر ورودی، کتاب‌ها را به ترتیب مرتب کند.

بخش سوم: مدیریت خطا (Exception)

در این بخش از شما خواسته شده که مطمئن شوید کد شما از خطاهای احتمالی به درستی جلوگیری کرده و هنگام مواجهه با خطاها پیام‌های مناسب نمایش دهد.

1. در متدهای **get_details** که در کلاس‌های **ScientificBook** و **FictionBook** ایجاد کرده‌اید، چک کنید که آیا تمام ویژگی‌ها به درستی مقداردهی شده‌اند یا خیر. در صورتی که هرکدام از ویژگی‌ها مقداردهی نشده باشد، یک **exception** با پیام خطای مناسب پرتاب کنید.

2. در متدهای **Extension** که برای بررسی موجود بودن کتاب و مرتب‌سازی ایجاد کرده‌اید، مدیریت خطای مناسبی را برای مواردی مانند ورودی‌های نادرست (مثلاً ورودی غیر از لیست کتاب‌ها) پیاده‌سازی کنید.

3. برای مدیریت خطا در زمان ایجاد اشیاء از کلاس‌ها (مثلاً کتاب‌های علمی یا داستانی)، اطمینان حاصل کنید که تمام داده‌های ورودی به درستی معتبر هستند. به عنوان مثال، سال انتشار باید یک عدد صحیح باشد و نوع کتاب باید یکی از انواع معتبر تعریف شده باشد.

بخش چهارم: ارزیابی و تست

در این بخش شما باید کدهای خود را برای اطمینان از صحت عملکردشان آزمایش کنید.

2. باغ اسرارآمیز و موجودات آن

ما یک باغ داریم که به صورت یک جدول دوبعدی ($m \times n$) نمایش داده می‌شود. هر خانه از این باغ می‌تواند خالی باشد یا در آن یک موجود زنده قرار بگیرد. دو نوع موجود زنده داریم: درخت (Tree) و حشره (Insect). همچنین برای مقابله با حشرات، ابزارهایی (Tool) در اختیار باغبان قرار می‌دهیم که یکی سم‌پاش (Sprayer) است و دیگری کوددهنده (Fertilizer). قرار است با یک منوی متنی عملیات زیر انجام شود:

1. افزودن موجود زنده (Add LivingBeing)
2. افزودن ابزار (Add Tool)
3. استفاده از ابزار (Use Tool)
4. محاسبه و نمایش ارزش کل باغ (CalculateGardenValue)
5. نمایش وضعیت باغ به صورت جدول (Show Garden)
6. خروج (Exit)

کلاس‌ها و فیلدهای مورد نیاز

1. کلاس پایه LivingBeing

- LivingBeing کلاس پایه‌ای است که تمامی موجودات زنده از آن ارث‌بری می‌کنند.
- فیلدهای زیر را دارد:
- `Name` (نام موجود)
- `Age` (سن موجود)
- `PositionX` (مختصات طول در باغ)
- `PositionY` (مختصات عرض در باغ)

- یک فهرست/لیستی سراسری از تمام موجودات زنده در باغ (برای اینکه بتوانیم در متدهای ارث‌برنده دسترسی داشته باشیم؛ البته فقط در کلاس‌های فرزند، دسترسی به این لیست فعال شود).

- نکته: مختصات `PositionX` و `PositionY` بعد از مقداردهی سازنده فقط خواندنی هستند (فقط getter دارند و setter قفل است).

- متد مجازی (با همان سطح دسترسی معمول) `CalculateValue`:
- در کلاس پایه، مقدار بازگشتی این متد برابر مجموع ارزش تمام موجودات زنده موجود در باغ است. (منظور: در کلاس پایه می‌توان یک متد ساده داشت که حلقه بزند روی همه موجودات در لیست و حاصل متد ویژه‌ی هرکدام را جمع بزند).
- در هر کلاس فرزند، این متد را بازنویسی (Override) می‌کنیم تا ارزش مربوط به همان موجود را برگرداند. اما وقتی از نمونه‌ی LivingBeing (یا لیست همه‌ی موجودات) متد `CalculateValue` را صدا می‌زنیم، باید مجموع ارزش تمام موجودات برگشت داده شود (با کمک پلی‌مورفیسم: متد هر موجودی متناسب با کلاسش فراخوانی می‌شود).

2. کلاس Tree (فرزند LivingBeing)

- علاوه بر فیلدهای به ارث‌رسیده، این فیلدها را دارد:
- `Height` (ارتفاع درخت)
- `FruitCount` (تعداد میوه)

- رفتار/قوانین:

- متد `CalculateValue` برای یک درخت، برابر است با:

$$(5 \times \text{FruitCount}) + (10 \times \text{Age}) = \text{Value}$$

مثلا اگر سن درخت ۵ باشد و ۱۰ میوه داشته

$$100 = 50 + 50 = 5 * 10 + 10 * 5$$

باشد، ارزشش برابر 100 خواهد بود.

- اگر $Height < 1$ باشد، باید خطایی در خروجی نشان دهیم (چاپ کنیم) و از افزودن آن جلوگیری کنیم.

- اگر $Age < 1$ یا $FruitCount < 0$ باشد هم باید خطای مناسب بدهیم.

- هنگام افزودن درخت به باغ، اگر مختصاتش قبلاً پر باشد، خطای "این خانه قبلاً اشغال شده است" بدهید.

3. کلاس Insect (فرزند LivingBeing)

- فیلدهای اختصاصی:

- $HarmLevel$ (میزان آسیب‌رسانی - یک عدد صحیح یا اعشاری)

- $VisionRange$ (بازه‌ی دید - اینکه حشره تا چند خانه در اطرافش را رصد می‌کند)

- رفتار/قوانین:

- متد $CalculateValue()$ برای یک حشره مقدار منفی در نظر گرفته می‌شود، زیرا حضور

حشره ارزش باغ را پایین می‌آورد. مثلاً:

$$Value = 1 - (HarmLevel \times Age)$$

یعنی اگر حشره ۳ سال عمر داشته باشد و

$HarmLevel=2$ باشد، مقدار آن 6- خواهد بود.

- اگر $VisionRange < 0$ یا $HarmLevel < 0$ بود خطای مناسب داده شود.

- اگر مختصات حشره را در یک خانه پر درخواست کنیم، خطای اشغال بودن خانه نشان داده شود.

4. کلاس پایه Tool

- این کلاس پایه برای ابزارهایی است که باغبان استفاده می‌کند و موارد زیر را دارد:
 - `ToolName` (نام ابزار)
 - `RemainingUses` (تعداد دفعاتی که می‌توان از ابزار استفاده کرد؛ هر بار استفاده یک واحد کم می‌شود)
 - لیستی سراسری از ابزارهای خریداری شده (مشابه موجودات زنده؛ برای مدیریت انتخاب ابزار و غیره).
- نکته: اگر `RemainingUses` پس از یک بار استفاده به ۰ برسد، دیگر اجازه‌ی استفاده‌ی مجدد از آن را نداریم (خطا نشان می‌دهیم).

5. کلاس Sprayer (فرزند Tool)

- فیلد اضافی:
- `SprayRange` (محدوده پاشش سم)
- رفتار:
- هنگامی که با این ابزار به یک خانه (x,y) شلیک/اسپری می‌کنیم، تمامی حشراتی که در محدوده $(x \pm \text{SprayRange}, y \pm \text{SprayRange})$ حضور دارند، از بین می‌روند (حذف می‌شوند). درخت‌ها آسیبی نمی‌بینند و تأثیری روی آن‌ها ندارد.
- اگر $\text{SprayRange} < 1$ باشد، خطای مناسب چاپ شود.
- هر بار استفاده از اسپری، `RemainingUses` یک واحد کم شود.

6. کلاس Fertilizer (فرزند Tool)

- فیلد اضافی:

- `FertilizerRange` (محدوده اثر کود)

- رفتار:

- وقتی از کود استفاده می‌کنیم و یک خانه هدف می‌گیرد (x,y) ، تمامی درخت‌هایی که در مربع محدوده‌ی $(x \pm \text{FertilizerRange}, y \pm \text{FertilizerRange})$ هستند، تعداد میوه‌شان ۲۰٪ افزایش پیدا می‌کند. (مثلاً اگر درختی ۱۰ میوه داشته باشد، می‌شود ۱۲).
- روی حشرات بی‌تأثیر است.
- اگر $\text{FertilizerRange} < 1$ باشد خطا بدهید.
- هر بار استفاده `RemainingUses` یک واحد کم می‌شود.

Extension Method

1. متدی بنام `<T>CountType` که برمی‌گرداند چند موجود از نوع `T` (مثلاً `Tree` یا `Insect`) در باغ وجود دارد.

منوی برنامه (روند اجرایی)

برنامه باید یک حلقه‌ی اصلی داشته باشد و تا زمانی که کاربر گزینه‌ی «خروج» را انتخاب نکرده است، ادامه پیدا کند. گزینه‌های منو:

1. Add LivingBeing

- یک خط بخوانید که بگوید نوع موجود چیست (مثلاً "Tree" یا "Insect")؟
- بسته به آن، فیلدهای لازم را گرفته و شیء بسازید:
- برای `Tree`: در یک خط و با فاصله، ``PositionY``، ``PositionX``، ``Age``، ``Name``، ``FruitCount``، ``Height``

- برای Insect: در یک خط و با فاصله، `PositionX` ، `PositionY` ، `Age` ، `Name` ، `VisionRange` ، `HarmLevel`
- بررسی کنید اگر مختصات خارج از محدوده‌ی ($m \times n$) است خطا بدهید.
- بررسی کنید اگر خانه پر است خطا بدهید.
- بررسی کنید اگر مقدار Height یا HarmLevel یا VisionRange یا Age نامعتبر است، خطا بدهید.
- در غیر این صورت، موجود را در لیست موجودات زنده ثبت و در خانه‌ی مورد نظر بگذارید.

2. Add Tool

- یک خط بخوانید که بگوید نوع ابزار چیست ("Sprayer" یا "Fertilizer")؟
- سپس در خط بعد، نام ابزار و تعداد دفعات قابل استفاده (RemainingUses) و اگر نیاز است مقدار محدوده (SprayRange / FertilizerRange).
- اگر نوع ابزار ناشناخته بود، پیام خطا بدهید.
- اگر $\text{RemainingUses} < 1$ بود، خطا بدهید.
- اگر محدوده‌ی ابزار > 1 بود، خطا بدهید.

3. Use Tool

- نام ابزار را گرفته، در لیست ابزارها جستجو کنید. اگر نبود، خطا. اگر RemainingUses تمام شده بود، خطا.
- اگر Sprayer است: مختصات هدف را بگیرید. اگر خارج از باغ بود، خطا. در محدوده SprayRange، حشرات را حذف کنید. RemainingUses را یک کم کنید.
- اگر Fertilizer است: مختصات هدف را بگیرید. در محدوده FertilizerRange، میوه‌ی درخت‌ها را ۲۰٪ افزایش دهید. RemainingUses را یک کم کنید.

4. CalculateGardenValue

- از طریق شیوه‌ای که در بالا گفتیم، روی کل موجودات زنده‌ی باغ متد `CalculateValue` را فراخوانی کنید و مجموع ارزش آن‌ها را بدست آورید و چاپ کنید.

5. Show Garden

- وضعیت تک تک خانه ها را به صورت یک جدول چاپ کنید:

- اگر خالی بود: `0`

- اگر درخت بود: `T`

- اگر حشره بود: `I`

6. Exit

- پایان برنامه.